

VULNERABILITY ASSESSMENT REPORT

OWASP Juice Shop – juice-shop.herokuapp.com

Prepared by: Pratik Giri | Date: April 27, 2026

Tools Used: Nmap 7.95 | OWASP ZAP 2.17.0 | Browser DevTools

Classification: CONFIDENTIAL

1. Executive Summary

This report presents the results of a vulnerability assessment conducted against the intentionally vulnerable OWASP Juice Shop web application hosted at juice-shop.herokuapp.com. The assessment was carried out on April 27, 2026, using Nmap for network reconnaissance and OWASP ZAP for web application scanning.

The target application was found to expose ports 80 (HTTP) and 443 (HTTPS), both served through a Heroku router. A total of 10 security alerts were identified through passive scanning, spanning medium, low, and informational risk levels. Additionally, during the assessment period, the application experienced intermittent availability issues returning HTTP 503 Service Unavailable responses.

No critical or high-severity vulnerabilities were discovered via passive scanning; however, the identified misconfigurations represent meaningful attack surface that should be remediated.

Risk Summary

Critical	High	Medium	Low
0	0	4	4

Additionally: 2 Informational findings were identified.

[illegible]

2. Scope & Methodology

2.1 Target

Application	OWASP Juice Shop (Intentionally Vulnerable)
URL	https://juice-shop.herokuapp.com
IP Address	54.220.192.176 (AWS eu-west-1)
Other IPs	54.73.53.134, 46.137.15.86
Assessment Date	April 27, 2026
Assessor	Pratik Giri (Kali Linux)

2.2 Methodology

The assessment followed a black-box approach using the following phases:

- Phase 1 – Network Reconnaissance: Nmap service version scan (-sV) to identify open ports, services, and software versions.
- Phase 2 – Web Application Scanning: OWASP ZAP 2.17.0 passive scan to enumerate security misconfigurations, missing headers, and information disclosures.
- Phase 3 – Manual Browser Analysis: Chrome DevTools Network tab to inspect HTTP response headers and status codes in real time.
- Phase 4 – Reporting: Aggregation, risk classification, and remediation recommendations.

3. Network Reconnaissance – Nmap Results

Command executed:

```
nmap -sV juice-shop.herokuapp.com
```

3.1 Open Ports

Port	State	Service	Version / Notes
80/tcp	Open	HTTP	heroku-router – Redirects to HTTPS
443/tcp	Open	SSL/HTTPS	heroku-router – Serves Juice Shop application

3.2 Key Findings

- 998 ports were found in a filtered state (no response), indicating firewall protection.
- The application is hosted on Heroku's AWS eu-west-1 infrastructure (PTR: ec2-54-220-192-176.eu-west-1.compute.amazonaws.com).
- The heroku-router service fingerprint was confirmed for both ports. Service fingerprints could not be fully matched to known signatures.
- Scan duration: 304.27 seconds for 1 host.

4. Web Application Scan – OWASP ZAP Findings

OWASP ZAP 2.17.0 was configured to scan juice-shop.herokuapp.com via passive analysis. A total of 10 alerts were identified across 5 vulnerability categories with medium and lower risk ratings.

4.1 Vulnerability Summary Table

#	Vulnerability	Risk	CWE	Confidence	WASC	Source
1	Content Security Policy (CSP) Header Not Set	Medium	693	High	15	Passive (10038)
2	Cross-Domain Misconfiguration	Medium	264	Medium	14	Passive (10098)
3	Missing Anti-clickjacking Header	Medium	1021	Medium	15	Passive (10020)
4	Session ID in URL Rewrite	Medium	200	High	13	Passive (10023)
5	Private IP Disclosure	Low	200	Medium	13	Passive (2)
6	Strict-Transport-Security Header Not Set	Low	319	High	15	Passive (10035)
7	Timestamp Disclosure - Unix	Low	200	Low	13	Passive (10096)
8	X-Content-Type-Options Header Missing	Low	693	Medium	15	Passive (10021)
9	Modern Web Application	Informational	N/A	Medium	N/A	Passive (10109)
10	Re-examine Cache-control Directives	Informational	524	Low	13	Passive (10015)

4.2 Detailed Vulnerability Analysis

Finding #1 – Content Security Policy (CSP) Header Not Set

Risk Level	Medium
CWE	CWE-693 – Protection Mechanism Failure
WASC ID	15
Confidence	High
Affected URL	https://juice-shop.herokuapp.com/
Source	Passive Scanner (Alert 10038)

Description:

The Content-Security-Policy (CSP) HTTP response header was absent from the application's responses. CSP is a defence-in-depth control that instructs browsers on which sources of content (scripts, styles, images, etc.) are trusted, significantly reducing the impact of Cross-Site Scripting (XSS) and data injection attacks.

Evidence:

ZAP confirmed the header was missing from GET https://juice-shop.herokuapp.com/ and associated resources including /ftp/eastere.gg, /ftp/encrypt.pyc, /ftp/package-lock.json.bak, and /sitemap.xml.

Remediation:

- Configure the web server, application server, or load balancer to include a Content-Security-Policy header in all responses.
- Start with a report-only policy (Content-Security-Policy-Report-Only) to capture violations before enforcing.
- Example: Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none';

Finding #2 – Cross-Domain Misconfiguration

Risk Level	Medium
CWE	CWE-264 – Permissions, Privileges, and Access Controls
WASC ID	14
Confidence	Medium
Affected URL	https://juice-shop.herokuapp.com/
Source	Passive Scanner

Description:

The response header Access-Control-Allow-Origin: * was observed in HTTP responses. This wildcard value permits any origin to make cross-domain requests to the application, enabling potential Cross-Origin Resource Sharing (CORS) abuse. Malicious sites could use this to read sensitive API responses if the user is authenticated.

Remediation:

- Replace the wildcard (*) with a specific allow-list of trusted origins.
- Do not combine Access-Control-Allow-Origin: * with Access-Control-Allow-Credentials: true, as this is a known CORS misconfiguration that can lead to credential theft.

Finding #3 – Missing Anti-Clickjacking Header

Risk Level	Medium
CWE	CWE-1021 – Improper Restriction of Rendered UI Layers
WASC ID	15
Confidence	Medium
Affected URL	https://juice-shop.herokuapp.com/ (2 instances)
Source	Passive Scanner (Alert 10020)

Description:

The application does not set the X-Frame-Options header or an equivalent frame-ancestors CSP directive. Without this protection, attackers can embed the application in an iframe on a malicious site and trick users into performing unintended actions (clickjacking).

Remediation:

- Add X-Frame-Options: DENY or X-Frame-Options: SAMEORIGIN to all responses.
- Alternatively, use the CSP frame-ancestors directive: Content-Security-Policy: frame-ancestors 'none';

Finding #4 – Session ID in URL Rewrite

Risk Level	Medium
CWE	CWE-200 – Exposure of Sensitive Information
WASC ID	13
Confidence	High
Source	Passive Scanner (Alert 10023)

Description:

Session identifiers were detected within URL parameters. Exposing session tokens in URLs risks them being captured in browser history, server access logs, referrer headers, and network proxies — all of which can enable session hijacking.

Remediation:

- Store session identifiers exclusively in cookies with the HttpOnly and Secure flags set.
- Never pass session tokens as URL query parameters.

Finding #5 – Private IP Disclosure

Risk Level	Low
CWE	CWE-200 – Exposure of Sensitive Information
WASC ID	13
Confidence	Medium
Affected Instances	2
Source	Passive Scanner

Description:

Private or internal IP addresses were found disclosed within HTTP responses. This information assists attackers in mapping internal infrastructure and can facilitate further attacks such as Server-Side Request Forgery (SSRF).

Remediation:

- Audit application responses, error messages, headers, and JavaScript files for any internal IP addresses.
- Remove or mask all internal network information before sending responses to clients.

Finding #6 – Strict-Transport-Security (HSTS) Header Not Set

Risk Level	Low
CWE	CWE-319 – Cleartext Transmission of Sensitive Information
WASC ID	15
Confidence	High
Source	Passive Scanner (Alert 10035)

Description:

The Strict-Transport-Security (HSTS) header was not present in HTTPS responses. Without HSTS, browsers may downgrade connections to HTTP upon subsequent visits, enabling man-in-the-middle attacks and SSL stripping.

Remediation:

- Add the header: Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
- Consider submitting the domain to the HSTS preload list at hstspreload.org.

Finding #7 – Timestamp Disclosure (Unix)

Risk Level	Low
CWE	CWE-200 – Exposure of Sensitive Information
WASC ID	13
Confidence	Low
Source	Passive Scanner (Alert 10096)

Description:

Unix timestamps were discovered within HTTP responses. While typically low severity, timestamp leakage can aid in fingerprinting server behaviour, correlating events, and refining attack timing.

Remediation:

- Avoid exposing server-side timestamps in responses.
- If timestamps are necessary for client functionality, use opaque tokens or relative time values.

Finding #8 – X-Content-Type-Options Header Missing

Risk Level	Low
CWE	CWE-693 – Protection Mechanism Failure
WASC ID	15
Confidence	Medium
Affected Instances	7
Source	Passive Scanner (Alert 10021)

Description:

The screenshot displays the OWASP ZAP (Zed Attack Proxy) interface. The top menu bar includes File, Edit, View, Analyse, Report, Tools, Import, Export, and Online Help. The main window is titled "Untitled Session - ZAP 2.17.0".

On the left sidebar, the "Standard Mode" tab is active. Below it, the "Sites" section shows a list of contexts, including "Default Context".

The main pane is divided into two sections. The top section shows a "Request" tab with a GET request to "http://www.any_domain_name.org/path HTTP/1.1". The request body is empty. Below the request, there's a "Response" tab. The bottom section shows a "Find" bar with "No matches" and a "Time: 0 ms Body Length: 0 Total Length: 0 bytes" status.

At the bottom, the "Alerts" section is visible, showing a list of alerts. The "Content Security Policy (CSP) Header Not Set (Systemic)" alert is selected. The alert details show a "Reference" to "https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html" and a "Alert Tags" table.

Key	Value
OWASP_2021_A05_POLICY_QA_STD	https://owasp.org/Top10/A05_2021_Security_Misconfiguration/
POLICY_PENTEST_SYSTEMIC_CWE-693	https://www.zaproxy.org/docs/desktop/addons/common-library/alerttags/#systemic

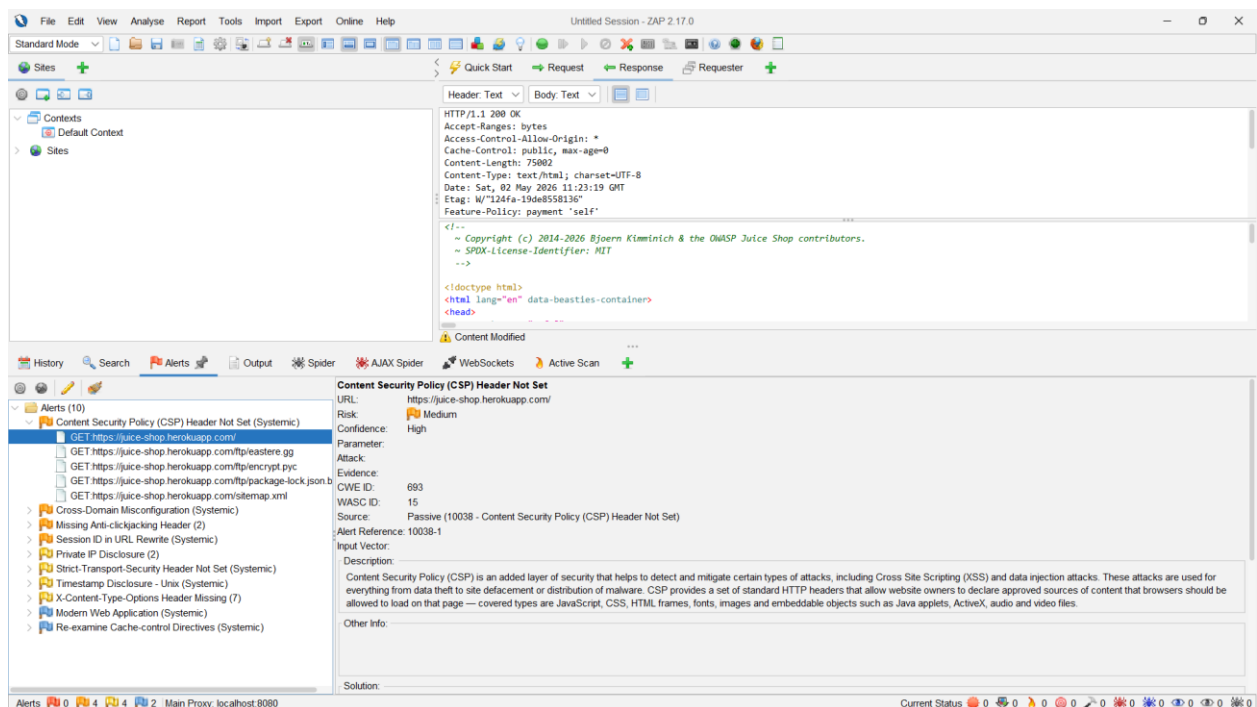
5. HTTP Status & Availability Observation

During manual browser-based testing, the application returned an HTTP 503 Service Unavailable response at 11:47:45 GMT on April 27, 2026. The response was served from the Heroku router (Remote Address: 54.73.53.134:443) with a Cache-Control: no-cache, no-store directive, confirming this was a genuine error and not a cached response.

Status Code	503 Service Unavailable
Timestamp	Sun, 27 Apr 2026 11:47:45 GMT
Server	Heroku
Via	1.1 heroku-router
Remote Address	54.73.53.134:443
Cache-Control	no-cache, no-store

The earlier ZAP passive scan at 11:23:19 GMT returned a successful HTTP/1.1 200 OK response, confirming the application was functional earlier in the assessment window. The 503 error is consistent with Heroku's free-tier dyno sleep behaviour or a transient infrastructure issue.

- Recommendation: Monitor application availability using an uptime service and configure auto-restart policies for the Heroku dyno.



6. Remediation Recommendations Summary

#	Finding	Priority	Action
1	CSP Header Not Set	Medium	Implement Content-Security-Policy header
2	CORS Misconfiguration	Medium	Restrict Access-Control-Allow-Origin to trusted origins
3	Missing Anti-Clickjacking	Medium	Add X-Frame-Options or CSP frame-ancestors
4	Session ID in URL	Medium	Use HttpOnly, Secure cookies for session tokens
5	Private IP Disclosure	Low	Remove internal IPs from all responses
6	HSTS Not Set	Low	Add Strict-Transport-Security header
7	Timestamp Disclosure	Low	Suppress server timestamps in responses
8	X-Content-Type-Options Missing	Low	Add X-Content-Type-Options: nosniff globally

The screenshot shows a web browser window with the URL `https://juice-shop.herokuapp.com`. The page displays an "Application error" message: "An error occurred in the application and your page could not be served. If you are the application owner, [check your logs for details](#). You can do this from the Heroku CLI with the command `heroku logs --tail`".

The browser's developer tools are open, showing the "Network" tab. A request to `https://juice-shop.herokuapp.com/` is selected, showing a 503 Service Unavailable status. The "Headers" sub-tab is active, displaying the following information:

- Request Headers:**
 - Request URL: `https://juice-shop.herokuapp.com/`
 - Request Method: `GET`
 - Status Code: `503 Service Unavailable`
 - Remote Address: `54.73.53.134:443`
 - Referrer Policy: `origin-when-cross-origin`
- Response Headers:**
 - Cache-Control: `no-cache, no-store`
 - Content-Length: `567`
 - Content-Type: `text/html; charset=utf-8`
 - Date: `Sat, 02 May 2026 11:47:45 GMT`
 - Nel: `["report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1]`
 - Report-To: `["group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=h7%2FjMQy0hUp5VMA1T5dZlCKMEFazyXGw4hqpPxkEzg0%3Du0026sid=812dcc77-0bd0-43b1-a5f1-b25750382959u0026ts=1777722461"}],"max_age":3600}]`
 - Reporting-Endpoints: `heroku-nel="https://nel.heroku.com/reports?s=h7%2FjMQy0hUp5VMA1T5dZlCKMEFazyXGw4hqpPxkEzg0%3Du0026sid=812dcc77-0bd0-43b1-a5f1-b25750382959u0026ts=1777722461"`
 - Server: `Heroku`
 - Via: `1.1 heroku-router`
- Request Headers:**
 - Accept: `text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7`
 - Accept-Encoding: `gzip, deflate, br, zstd`

The conclusion section summarizes that the assessment found **10 security findings** — no critical/high issues via passive scanning, but 4 medium-severity misconfigurations (missing CSP, CORS wildcard, no clickjacking protection, session ID in URL) that are all fixable through server-side configuration. It notes Juice Shop is intentionally vulnerable (a training app), and flags that **active scanning would likely uncover deeper flaws** like SQL Injection and Broken Authentication. The final recommendations are to deploy CSP, restrict CORS, enable HSTS, secure session handling, and scrub internal IPs from responses

The screenshot shows the Chrome DevTools Network tab with a request to `juice-shop.herokuapp.com`. The response is a 503 Service Unavailable status code. The response headers are visible, including Cache-Control, Content-Length, Content-Type, Date, Nel, Report-To, Reporting-Endpoints, Server, and Via. The request headers are also visible, including Accept and Accept-Encoding.

Header	Value
Request URL	https://juice-shop.herokuapp.com/
Request Method	GET
Status Code	503 Service Unavailable
Remote Address	54.73.53.134:443
Referrer Policy	origin-when-cross-origin
Cache-Control	no-cache, no-store
Content-Length	567
Content-Type	text/html; charset=utf-8
Date	Sat, 02 May 2026 11:47:45 GMT
Nel	{ "report_to": "heroku-nel", "response_headers": ["Via"], "max_age": 3600, "success_fraction": 0.01, "failure_fraction": 0.1 }
Report-To	{ "group": "heroku-nel", "endpoints": [{ "url": "https://nel.heroku.com/reports?s=h7%2FjMQy0hUp5VMA1TSjdZICKMEFazyXGw4hqpPxZg0%3D\u0026sid=812dcc77-0bd0-43b1-a5f1-b25750382959\u0026ts=177722461" }], "max_age": 3600 }
Reporting-Endpoints	heroku-nel="https://nel.heroku.com/reports?s=h7%2FjMQy0hUp5VMA1TSjdZICKMEFazyXGw4hqpPxZg0%3D&sid=812dcc77-0bd0-43b1-a5f1-b25750382959&ts=177722461"
Server	Heroku
Via	1.1 heroku-router
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding	gzip, deflate, br, zstd

7. Conclusion

The vulnerability assessment of OWASP Juice Shop (juice-shop.herokuapp.com) identified 10 security findings. No critical or high-severity vulnerabilities were discovered through passive scanning; the most significant issues relate to missing security headers and CORS misconfiguration — all of which are straightforward to remediate through server-side configuration changes.

It is important to note that OWASP Juice Shop is an intentionally vulnerable application designed for security training. In a production environment, the same class of findings could represent serious risks including Cross-Site Scripting, clickjacking, session hijacking, and information disclosure.

The following immediate actions are recommended:

- Deploy a comprehensive Content-Security-Policy header.
- Restrict CORS to trusted origins only.
- Enable HSTS and anti-clickjacking protections.
- Ensure session tokens are never transmitted via URL parameters.
- Remove all internal infrastructure information from HTTP responses.

Active scanning (not performed in this assessment) would likely reveal additional application-layer vulnerabilities such as SQL Injection, Broken Authentication, and Insecure Direct Object References — all well-documented weaknesses in the Juice Shop application.